You're reading the documentation for an older, but still supported, version of ROS 2. For information on the latest version, please have a look at Kilted.

Writing a Hardware Component

In ros2_control hardware system components are libraries, dynamically loaded by the controller manager using the pluginlib interface. The following is a step-by-step guide to create source files, basic tests, and compile rules for a new hardware interface.

1. **Preparing package**

   If the package for the hardware interface does not exist, then create it first. The package should have `ament_cmake` as a build type. The easiest way is to search online for the most recent manual. A helpful command to support this process is `ros2 pkg create`. Use the `--help` flag for more information on how to use it. There is also an option to create library source files and compile rules to help you in the following steps.

2. **Preparing source files**

   After creating the package, you should have at least `CMakeLists.txt` and `package.xml` files in it. Create also `include/<PACKAGE_NAME>/` and `src` folders if they do not already exist. In `include/<PACKAGE_NAME>/` folder add `<robot_hardware_interface_name>.hpp` and `<robot_hardware_interface_name>.cpp` in the `src` folder. Optionally add `visibility_control.h` with the definition of export rules for Windows. You can copy this file from an existing controller package and change the name prefix to the `<PACKAGE_NAME>`.

3. **Adding declarations into header file (.hpp)**

   1. Take care that you use header guards. ROS2-style is using `#ifndef` and `#define` preprocessor directives. (For more information on this, a search engine is your friend :)).
   2. Include `"hardware_interface/$interface_type$_interface.hpp"` and `visibility_control.h` if you are using one. `$interface_type$` can be `Actuator`, `Sensor` or `System` depending on the type of hardware you are using. for more details about each type check Hardware Components description.
   3. Define a unique namespace for your hardware_interface. This is usually the package name written in `snake_case`.
   4. Define the class of the hardware_interface, extending `$InterfaceType$Interface`, e.g., .. code:: c++ class HardwareInterfaceName : public hardware_interface::$InterfaceType$Interface

5. Add a constructor without parameters and the following public methods implementing `LifecycleNodeInterface`: `on_configure`, `on_cleanup`, `on_shutdown`, `on_activate`, `on_deactivate`, `on_error`; and overriding `$InterfaceType$Interface` definition: `on_init`, `export_state_interfaces`, `export_command_interfaces`, `prepare_command_mode_switch` (optional), `perform_command_mode_switch` (optional), `read`, `write`. For further explanation of hardware-lifecycle check the pull request and for exact definitions of methods check the `"hardware_interface/$interface_type$_interface.hpp"` header or doxygen documentation for *Actuator*, *Sensor* or *System*.

4. **Adding definitions into source file (.cpp)**

    1. Include the header file of your hardware interface and add a namespace definition to simplify further development.
    2. Implement `on_init` method. Here, you should initialize all member variables and process the parameters from the `info` argument. In the first line usually the parents `on_init` is called to process standard values, like name. This is done using: `hardware_interface::(Actuator|Sensor|System)Interface::on_init(info)`. If all required parameters are set and valid and everything works fine return `CallbackReturn::SUCCESS` or `return CallbackReturn::ERROR` otherwise.

    4. Write the `on_configure` method where you usually setup the communication to the hardware and set everything up so that the hardware can be activated.
    5. Implement `on_cleanup` method, which does the opposite of `on_configure`.
    6. Implement `export_state_interfaces` and `export_command_interfaces` methods where interfaces that hardware offers are defined. For the `Sensor`-type hardware interface there is no `export_command_interfaces` method. As a reminder, the full interface names have structure `<joint_name>/<interface_type>`.
    7. (optional) For *Actuator* and *System* types of hardware interface implement `prepare_command_mode_switch` and `perform_command_mode_switch` if your hardware accepts multiple control modes.
    8. Implement the `on_activate` method where hardware “power” is enabled.
    9. Implement the `on_deactivate` method, which does the opposite of `on_activate`.
    10. Implement `on_shutdown` method where hardware is shutdown gracefully.
    11. Implement `on_error` method where different errors from all states are handled.
    12. Implement the `read` method getting the states from the hardware and storing them to internal variables defined in `export_state_interfaces`.
    13. Implement `write` method that commands the hardware based on the values stored in internal variables defined in `export_command_interfaces`.
    14. IMPORTANT: At the end of your file after the namespace is closed, add the `PLUGINLIB_EXPORT_CLASS` macro.

For this you will need to include the `"pluginlib/class_list_macros.hpp"` header. As first parameters you should provide exact hardware interface class, e.g., `<my_hardware_interface_package>::<RobotHardwareInterfaceName>`, and as second the base class, i.e., `hardware_interface::(Actuator|Sensor|System)Interface`.

5. **Writing export definition for pluginlib**

    1. Create the `<my_hardware_interface_package>.xml` file in the package and add a definition of the library and hardware interface's class which has to be visible for the pluginlib. The easiest way to do that is to check definition for mock components in the hardware_interface mock_components section.
    2. Usually, the plugin name is defined by the package (namespace) and the class name, e.g., `<my_hardware_interface_package>/<RobotHardwareInterfaceName>`. This name defines the hardware interface's type when the resource manager searches for it. The other two parameters have to correspond to the definition done in the macro at the bottom of the `<robot_hardware_interface_name>.cpp` file.

6. **Writing a simple test to check if the controller can be found and loaded**

    1. Create the folder `test` in your package, if it does not exist already, and add a file named `test_load_<robot_hardware_interface_name>.cpp`.
    2. You can copy the `load_generic_system_2dof` content defined in the test_generic_system.cpp package.
    3. Change the name of the copied test and in the last line, where hardware interface type is specified put the name defined in `<my_hardware_interface_package>.xml` file, e.g., `<my_hardware_interface_package>/<RobotHardwareInterfaceName>`.

7. **Add compile directives into ``CMakeLists.txt`` file**

    1. Under the line `find_package(ament_cmake REQUIRED)` add further dependencies. Those are at least: `hardware_interface`, `pluginlib`, `rclcpp` and `rclcpp_lifecycle`.
    2. Add a compile directive for a shared library providing the `<robot_hardware_interface_name>.cpp` file as the source.
    3. Add targeted include directories for the library. This is usually only `include`.
    4. Add ament dependencies needed by the library. You should add at least those listed under 1.
    5. Export for pluginlib description file using the following command: .. code:: cmake

        pluginlib_export_plugin_description_file(hardware_interface <my_hardware_interface_package>.xml)

    6. Add install directives for targets and include directory.
    7. In the test section add the following dependencies: `ament_cmake_gmock`, `hardware_interface`.

8. Add compile definitions for the tests using the `ament_add_gmock` directive. For details, see how it is done for mock hardware in the ros2_control package.
9. (optional) Add your hardware interface`s library into `ament_export_libraries` before `ament_package()` .

8. **Add dependencies into ``package.xml`` file**

    1. Add at least the following packages into `<depend>` tag: `hardware_interface` , `pluginlib` , `rclcpp` , and `rclcpp_lifecycle` .
    2. Add at least the following package into `<test_depend>` tag: `ament_add_gmock` and `hardware_interface` .

9. **Compiling and testing the hardware component**

    1. Now everything is ready to compile the hardware component using the `colcon build <my_hardware_interface_package>` command. Remember to go into the root of your workspace before executing this command.
    2. If compilation was successful, source the `setup.bash` file from the install folder and execute `colcon test <my_hardware_interface_package>` to check if the new controller can be found through `pluginlib` library and be loaded by the controller manager.

That's it! Enjoy writing great controllers!

Useful External References

- Templates and scripts for generating controllers shell

    Note

    The script is currently only recommended to use with Foxy and Humble, not compatible with the API from Jazzy and onwards.